Track - A:
Maths & Stats

Part-1:

Linear Algebra – I

LET'S DIVE IN ▶ ▶ ▶

Context

AI creates everything and understands nothing

Text | Code | Images | Music | Video

AI has evolved to generate all of these.

But here's the paradox — AI neither understands human emotions, nor grasps the meaning of the text or code it is producing.
So how does it do it? — AI, as a system, understands exactly one thing: numbers. Through number manipulation, it does everything above.

The question this deck answers:
what are those numbers,
and what does AI actually do with them?

The one-sentence version

Everything becomes vectors. Then AI makes two moves.

Modern AI turns **things** (words, sentences, images, users, products) into **lists of numbers (vectors)**, then performs two operations on them, over and over:

1

Measuring angles between them

= similarity. Is this document like that query? Is this user like that buyer? Is this token relevant to that one?

2

Transforming them

= matrix multiplication. Reshape the vector, layer after layer, until the answer is easy to read off.

Everything in this deck is the vocabulary for those two moves.

Roadmap

Five concepts, one PM lens

1 **The data containers**
Scalars → Vectors → Matrices → Tensors, shapes, transpose, y = Wx + b

2 **The dot product**
The most important operation in AI — attention, retrieval, recommendations

3 **Cosine vs Euclidean**
Two ways to say “close” — and why text search picks cosine

4 **Matrix multiplication**
The engine room — deep networks, GPUs, and your API bill

5 **The coordinate system**
Basis, vector spaces, orthogonality, linear independence

\+ **The PM layer**
Misconceptions to avoid, why it matters, interview prep with model answers

1

The Data Containers

Scalars → *Vectors* → *Matrices* → *Tensors.*
Think of these as boxes of increasing dimension, same numbers, more structure.

Concept 1 · Overview

The ladder: same numbers, more structure

Rank-0

Scalar

One number.

```
23.5
```

Rank-1

Vector

An ordered list of numbers.

```
[9, 1]
```

Rank-2

Matrix

A 2-D grid of numbers.

```
[[1,2],
 [3,4]]
```

Rank-3+

Tensor

The general n-D case — “more axes.”

```
(8×2048×4096)
```

Each step up adds an axis.
A vector is a stack of scalars;
a matrix is a stack of vectors;
a tensor is a stack of matrices.

Scalar: one number (Rank-0)

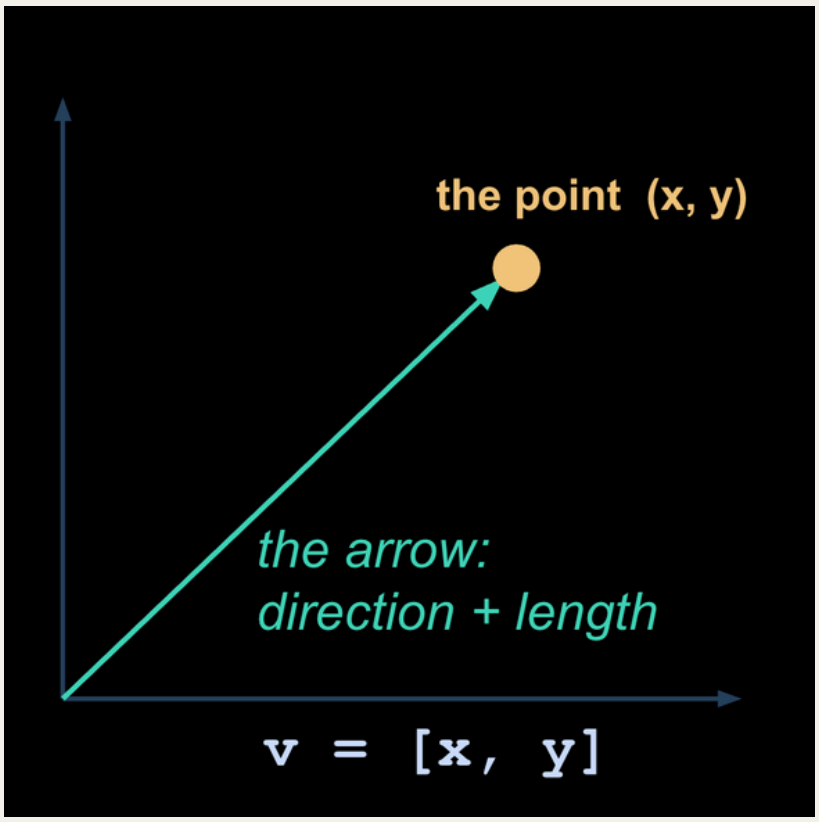
The simplest container: a single value, no structure.

`23.5`	temperature
`0.87`	a model's confidence
`₹499`	price

Scalars are where the story starts,
but nothing interesting in AI happens
until numbers come in ordered lists.

Vector:
an ordered list of numbers (Rank-1)

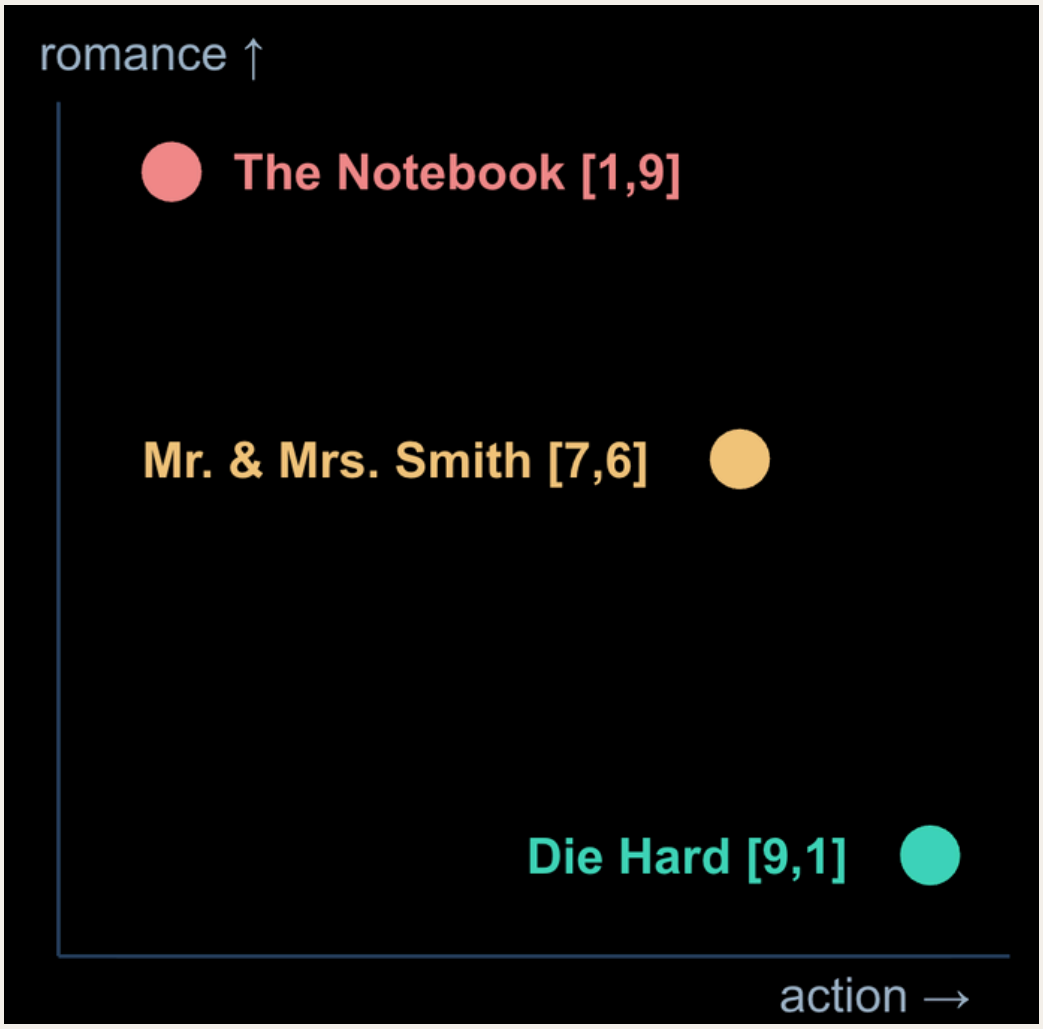
The key mental shift: a vector is simultaneously a POINT in space (coordinates) and an ARROW from the origin to that point (direction + length).
Both views matter: the point view gives you “nearby = similar”; the arrow view gives you direction, angle and alignment.
In n dimensions: a list of n numbers = one location in n-dimensional space. Two numbers = a 2-D map; 1,536 numbers = a 1,536-D map. Same idea.



Movies as vectors: [action, romance]

```
Die Hard          = [9, 1]   ← almost pure action
The Notebook      = [1, 9]   ← almost pure romance
Mr. & Mrs. Smith = [7, 6]   ← both
```

Each movie is now a point on a 2-D map
and “similar movies” = “nearby points.”
You'll reuse this all week
for the dot product, similarity, and recommendations.



Concept 1 · Vector — the real thing

That's the entire idea of embeddings

- **From 2 dimensions to 512–3072**
  - real embedding models use 512–3072 dimensions instead of 2
  - and LEARN the dimensions from data instead of us naming them.
- **A sentence embedding is exactly this**
  - a list of ~1,536 numbers that locates the sentence's MEANING in space.
  - “text-embedding” models output these.
- **Second example — a customer as a vector**
  - [avg_order_value, orders_per_month, months_since_signup] → customer_A = [1200, 3, 14].
- **This pattern is everywhere**
  - every ML system that “profiles” users is building vectors like this.

Takeaway: “similar things sit near each other in vector space” is the single most reused idea in modern AI.

Matrix: a 2-D grid of numbers (Rank-2)

Two ways you'll meet it:

1 · A stack of vectors (data)

10 movies × 2 dimensions = a (10×2) matrix. A batch of 1,000 document embeddings at 1,536 dims = a (1000×1536) matrix.

Rows = items, columns = dimensions.

2 · A transformation (machine)

A neural-network layer's weights ARE a matrix — a machine that eats a vector and outputs a new vector.

Details in Concept 4 (matrix multiplication).

Same object, two roles:
data at rest, or a machine that transforms it.

Concept 1 · Tensor

Tensor: the general n-D case (Rank-3 and up)

Just “more axes”:

A color image — 3-D

`(height × width × 3 RGB channels)`

A BATCH of images — 4-D

`(batch × height × width × channels)`

LLM activations — 3-D

`(batch × sequence_length × hidden_dim) — e.g., (8 × 2048 × 4096)`

PM hook: when engineers discuss “activation memory,” this object, the activation tensor, is what's filling the GPU.

Shapes: the #1 practical skill

- **Every ML debugging conversation starts here**
  - “what's the shape?”
  - Notation: (rows × columns).
- **The rule:**
  - inner numbers must match
  - they “cancel.”
- **“Dimension Mismatch” decoded**:
  - When an engineer says “dimension mismatch,” this is all they mean: two boxes whose edges don't line up.

```
(3×4)  ·  (4×2)  →  (3×2)    ✓
  inner numbers match (4 = 4)

(3×4)  ·  (2×4)  →  ERROR    ✗
  inner numbers don't match (4 ≠ 2)
```

Read it aloud: “three-by-four times four-by-two gives three-by-two.” The 4s cancel; the outer numbers remain.

Concept 1 · Transpose & the famous equation

Transpose, and reading y = Wx + b

Transpose (A^T) flips rows ↔ columns
a (3×4) becomes (4×3).

Convention matters (row vs column vectors); transpose exists mostly to make shapes line up.

$$y = Wx + b$$

the most famous equation in deep learning

How to read it: “take input vector x, transform it with weight matrix W, shift it by bias vector b, get output y.”

Get comfortable reading this, it appears everywhere.

Next slide: a tiny concrete one, worked by hand.

Concept 1 · Worked example

y = Wx + b, by hand

Given:
W = [[1, 0, 2], (2×3 weights)
[0, 1, 1]]

x = [10, 20, 30] (input, 3×1)

b = [1, 5] (bias, 2×1)

Step 1 — Wx (each row of W dot x):
row 1: 1·10 + 0·20 + 2·30 = 70
row 2: 0·10 + 1·20 + 1·30 = 50
Wx = [70, 50]

Step 2 — add b:
y = Wx + b = [70+1, 50+5] = [71, 55]

Shapes: (2×3)·(3×1) + (2×1) → (2×1).
A 3-number input became a 2-number output.
That's what one neural layer does, millions of times, with learned W.

2

The Dot Product

The most important operation in AI.
One formula, two faces, three places you'll meet it.

Concept 2 · The formula

One operation, two equivalent faces

ALGEBRA

$$a \cdot b = \Sigma\, a_i b_i$$

multiply matching positions, add up.
Just arithmetic — multiply and add.

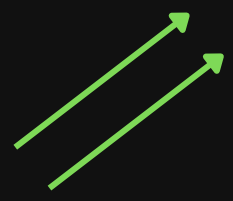
GEOMETRY

$$a \cdot b = |a||b|\cos(\theta)$$

lengths × angle-alignment.
The same number, seen as geometry.

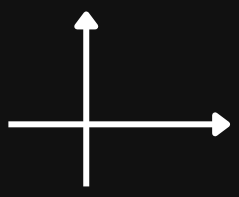
The intuition: the dot product is an ALIGNMENT SCORE between two arrows.

An alignment score: three regimes



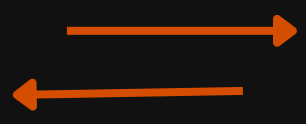
Positive

pointing the same way (similar)



Zero

perpendicular — unrelated, “orthogonal”



Negative

pointing in opposing directions (dissimilar)

Physical analogy: pushing a shopping cart:

- push in the direction it faces → full effect (max dot product).
- Push sideways → nothing (zero).
- Push backwards → negative.

The dot product measures “how much of your effort counts.”

Concept 2 · Worked example

The movies again — multiply and add

```
Die Hard · Mr.&Mrs.Smith
   = [9,1]·[7,6]
   = 63 + 6 = 69
   high: both action-heavy

Die Hard · The Notebook
   = [9,1]·[1,9]
   =  9 + 9 = 18
    low: different genres
```

The numbers behave like a similarity ranking:

- higher dot product = more alike, on the dimensions the vectors encode.

No magic:

- multiply and add.

That's the whole operation — done trillions of times a day across AI systems.

Concept 2 · Where you'll meet it — 1 of 3

Attention IS dot products

Inside a transformer:

- every token asks “who's relevant to me?” by taking dot products between its Query vector and all Key vectors.

Query · Key = relevance score:

- those scores decide which other tokens each token “attends” to — attention is literally an alignment ranking.

At scale:

- GPT computing attention over a 2,000-token context = millions of dot products PER LAYER.

Preview: Week 4 opens the transformer and you'll see these Q·K scores up close. Today you already know the core operation.

Retrieval IS dot products

The vector DB's job:

- answering “find documents similar to this query” = compute query · document for candidates, then rank.

Your RAG pipeline's “relevance”

- is this number.
- When retrieval quality is debated, this score is what's being debated.

Same operation, new name

- “semantic search,” “similarity search,” “nearest neighbors”, all rankings of dot products (or their normalized cousin, next section).

PM translation: “the bot retrieved the wrong document” usually means “the wrong vectors scored highest.”

Recommendations ARE dot products

`user_vector · item_vector = predicted affinity`

```
user = [loves_action = 8,
        loves_romance = 2]

user · Die Hard.     = 8·9 + 2·1 = 74 ← recommend
user · The Notebook = 8·1 + 2·9 = 26
```

Recommend Die Hard:

- the highest dot product wins.

Netflix-style matrix factorization is this at scale:

- learn a vector per user and per item, recommend by dot product,

This is the pre-LLM workhorse that still runs large parts of the internet.

3

Cosine Similarity vs Euclidean Distance

Two ways to say “close.”
One cares about direction,
the other about distance,
and for text, the difference is everything.

Direction vs distance

Cosine similarity

$$\cos(a,b) = (a \cdot b) / (|a||b|)$$

= the dot product after ignoring lengths = cos(θ)
Range: [−1, +1]
Answers: “do these arrows point the same way?”
Direction only — magnitude ignored.
$|a| = \sqrt{(\Sigma a_i^2)}$ is the length.

Euclidean (L2) distance

$$L2(a,b) = \sqrt{(\Sigma(a_i - b_i)^2)}$$

= straight-line distance between the two POINTS
Range: [0, ∞)
Answers: “how far apart are they?”
Magnitude matters — length is part of the answer.

Do this by hand once, the classic

```
a = [1, 2, 2],  b = [2, 1, 2]

a·b = 1·2 + 2·1 + 2·2 = 8
|a| = √(1+4+4) = 3,
|b| = √(4+1+4) = 3

cosine = 8 / (3·3) = 8/9 ≈ 0.89
       → highly similar
```

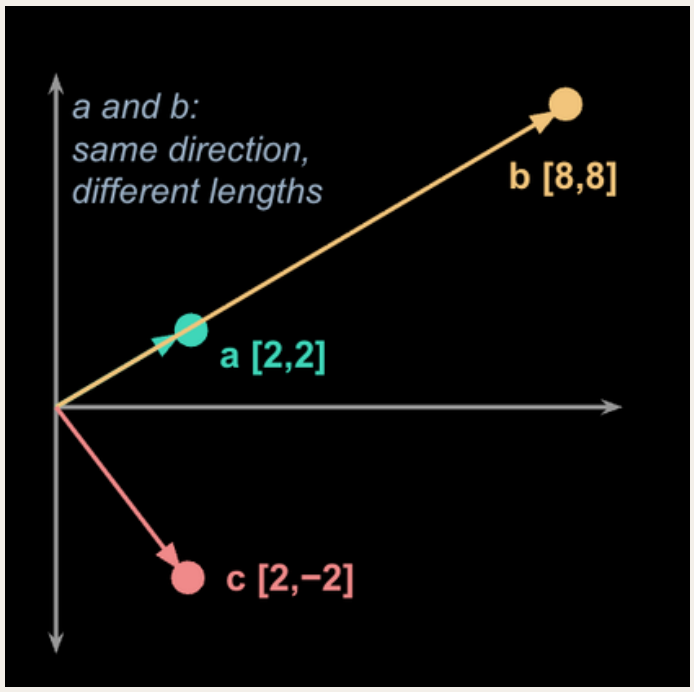
Three multiplications,
two square roots,
one division.

Every "semantic similarity score" you've ever seen in a demo is a number like this one.

The example that makes it click

```
a = [2, 2]     ← SHORT doc about "pricing"
b = [8, 8]     ← LONG doc about "pricing"
                   (same topic, 4× intensity)
c = [2, -2]    ← doc about something else entirely
```

```
cosine(a,b) = 1.0
              identical direction — same topic!
L2(a,b)     = √72 ≈ 8.49
              far apart — because b is "bigger"
cosine(a,c) = 0.0
              orthogonal — unrelated
L2(a,c)     = 4.0
              …closer than b!  misleading
```



Concept 3 · The lesson

Why text search uses cosine

- **Euclidean makes the unrelated docc look CLOSER to a than same-topic b**
  - length polluted the answer.
- **Text search wants “same topic,” not “same length”**
  - direction encodes topic/meaning;
  - magnitude often just reflects document length or term frequency.
  - That is exactly why text search uses cosine.
- **Where Euclidean DOES belong**
  - wherever magnitude is meaningful
  - clustering physical measurements

Rule of thumb:
meaning → cosine.
measurements → euclidean.

Concept 3 · Principal-level detail

Normalization: the fake-debate insurance

Normalized embeddings

- rescaled to length 1 — most modern embedding APIs do this by default.

The consequence

- cosine ranking ≡ dot-product ranking ≡ Euclidean ranking.
- All three give the same order.

The payoff

- if you hear a team debating “cosine vs dot product” over normalized vectors,
- the debate is empty,
- and you're the person who can say so.

PM implication: when your engineer says “we use cosine over L2,” they mean “we care about semantic direction, not vector length.” Nod — and ask whether the embeddings are normalized.

4

Matrix Multiplication

The engine room.
Every output cell is one dot product.
And your API bill is, to first order,
a matrix-multiplication bill.

Every output cell = one dot product

The rule

- each cell of the output = (row of A) · (column of B).

Shapes & cost

- (m×n)·(n×p) → (m×p),
- costing m·n·p multiply-adds
- this product of three numbers is where compute bills come from.

```
A = [[1, 2],      B = [[5, 6],
     [3, 4]]           [7, 8]]

AB = [[1·5+2·7, 1·6+2·8],    [[19, 22],
      [3·5+4·7, 3·6+4·8]] =   [43, 50]]
```

Top-left cell 19 = row [1,2] · column [5,7] = 5 + 14.
Four cells, four dot products.

Concept 4 · Two mental models (both true)

A machine that reshapes space and chains

1 · Transformation

a matrix is a machine that reshapes space — rotate, stretch, squash, shear. Multiplying a vector by a matrix moves the point somewhere new.

Watch 3Blue1Brown, Essence of Linear Algebra

2 · Composition

multiplying two matrices chains two transformations into one — "rotate, THEN stretch." THIS IS WHAT A DEEP NETWORK IS: layer after layer of transform, until the answer is easy to read off.

"Deep" = many chained transformations.

An image or a sentence enters as a vector;
each layer reshapes it;
the answer falls out at the end.

Concept 4 · Scale check

Why this is the whole game

Billions — of weights in GPT-class models — organized as matrices

Hundreds — of large matrix multiplications to generate ONE token

≈ Your API bill — is, to first order, a matrix-multiplication bill

When you negotiate model choice, context length, or latency budgets, you are negotiating how many of these multiplications happen and on whose hardware.

Concept 4 · The punchline

Why GPUs — and why NVIDIA

Each output cell is an INDEPENDENT dot product

- thousands of them, none waiting on another — a perfectly parallel workload.

GPUs have thousands of small cores

- designed to do exactly this in parallel. They were built to transform 3-D game geometry — same math.

CPUs have a few dozen powerful cores

- optimized for sequential work. On this workload, GPUs win by 10–100×.

This single fact explains NVIDIA's position in AI.
(The full origin story is this week's Track E: AlexNet — trained on two gaming GPUs in 2012.)

The “do nothing” and the “undo”

Identity matrix I — the “do nothing” transformation

- 1s on the diagonal: AI = A. The number 1 of the matrix world.

Inverse A^{-1} — the “undo” button

- **$A^{-1}A = I$**. If A doubles every vector, A^{-1} halves it. Only square, non-degenerate matrices have one.

Not everything can be undone

- a matrix that flattens 3-D space onto a plane DESTROYED the third dimension
- no inverse exists.

“Non-invertible = information was lost.” You'll never invert a matrix as a PM, but the metaphor is genuinely useful: lossy compression, dimensionality reduction, irreversible aggregations, and later, quantization and embedding-privacy questions.

5

The Coordinate System

Basis, vector spaces, orthogonality, linear independence, the vocabulary of what those 1,536 dimensions actually are.

The world — and its axes

Vector space — everything you can reach by adding and scaling vectors. The “world” the vectors live in.
Basis — the minimal set of reference directions that describes the whole space — the axes of your coordinate system.
Embedding dimension 1536 — the model describes any MEANING as a mix of 1,536 learned directions.

THE ANALOGY THAT LANDS: RGB COLOR

Every color your screen shows is some mix of (red, green, blue). Three basis directions; (r,g,b) coordinates describe any color. An embedding model is a color system for semantics, with 1,536 “primary colors” — where nobody hand-named the axes.

No redundant directions

No vector in your set is redundant,
none can be built from the others.

DEPENDENT (redundant)

`[price_usd, price_inr]`

one is a fixed multiple of the other — the second column adds cost, zero information.

INDEPENDENT

`[price, weight]`

knowing one doesn't give you the other — each direction carries new information.

Redundant dimensions = paying storage and compute for nothing.

Perpendicular = uncorrelated directions

Definition

- perpendicular; dot product = 0; “fully uncorrelated directions.”

Everyday example

- latitude and longitude are orthogonal — moving north tells you nothing about east.

Why it matters in embedding spaces

- roughly-orthogonal directions are how a model packs MANY independent concepts (tense, sentiment, topic, formality…) into one space without them interfering.

Bonus — superposition

- in high dimensions there's room for NEARLY-orthogonal directions vastly exceeding the dimension count — one reason 1,536 numbers can encode so much.
- See Anthropic's interpretability work.

```
Ref (rabbit hole): “Toy Models of Superposition” — transformer-circuits.pub/2022/toy_model/index.html
```

Concept 5 · Payoff

The conversation this vocabulary unlocks

YOU

"Vector DB cost is scaling with dimensions. Can we cut dimensions without hurting retrieval quality?"

ENGINEER

"We can try Matryoshka embeddings — truncate 1536 → 512 — or PCA."

YOU

"So we'd keep the most informative directions and drop near-redundant ones. What recall do we lose at 512? Can we A/B it on the golden set?"

Cost lever + technique + measured trade-off.

That exchange is AIPM-level,
and it's just Concept 5 plus Week 5's evals.

Pre-empt these (part 1)

1

MYTH *“Embedding dimensions are human concepts like ‘happiness.’”*

TRUTH No — they're learned, distributed, and individually meaningless; only directions/regions have meaning.

2

MYTH *“Higher cosine similarity always means better retrieval.”*

TRUTH It means nearer in the embedding model's opinion — if the model wasn't trained for your domain, near can be wrong (Week 8: domain shift, rerankers).

Pre-empt these (part 2)

3

MYTH *"A 3072-dim embedding is twice as good as 1536."*

TRUTH More dims = more capacity AND more cost; quality depends on the model, not the count.

4

MYTH *"The matrix math is what's hard about AI."*

TRUTH The operations are multiply-and-add, learnable in a week; the hard parts are data, evals, and product judgment — the rest of this program.

PM perspective

Why this topic matters for PMs

Embeddings are vectors

- every semantic feature you ship — search, dedup, clustering, recommendations, RAG — rests on vector geometry.
- When quality disappoints, the geometry (embedding model choice, normalization, domain fit) is suspect #1.

Similarity search depends on cosine similarity

- “retrieval brought back garbage” often decomposes into: wrong embedding model for the domain, unnormalized vectors, or chunks whose vectors blur multiple topics (Week 8).

Recommendation systems compare vectors

- user · item = affinity — matrix factorization, the pre-LLM workhorse that still runs large parts of the internet.

RAG retrieves nearby embeddings

- “the bot answered from the wrong document” often = “the nearest vectors weren't the right vectors.” You can now ask WHICH — the query's embedding, the docs', or the distance metric?

Cost conversations are shape conversations

- tokens × dimensions × layers = compute. The vocabulary of shapes lets you follow — and interrogate — every infra cost discussion in Weeks 4–12.

The through-line: you don't need to DO this math at work. You need to recognize which of the two moves, similarity or transformation, your product's problem lives in, and ask cost- and quality-shaped questions about it.

Say it like this

Q

What is cosine similarity, and why is it preferred over Euclidean distance for text embeddings?

MODEL ANSWER

"Cosine measures the angle between two vectors — direction only — while Euclidean measures straight-line distance, which mixes in magnitude. For text, direction encodes topic/meaning and magnitude often just reflects length or intensity, so two documents about the same topic at different lengths score as similar under cosine but far apart under L2. That's why semantic search defaults to cosine."

Bonus: for normalized embeddings the two rankings coincide.

Say it like this

Q

Explain the dot product to a non-technical stakeholder.

MODEL ANSWER

“It's an alignment score between two arrows: big positive = pointing the same way, zero = unrelated, negative = opposite. Like pushing a shopping cart — only the push in the direction it faces counts.”

Say it like this

Q **Why are GPUs good at deep learning?**

MODEL ANSWER

“Deep learning is overwhelmingly matrix multiplication, and every cell of a matrix multiply is an independent dot product — thousands of small jobs with no dependencies. GPUs have thousands of cores built for exactly that parallel pattern; CPUs have a few powerful cores optimized for sequential work. Same reason they were good at 3-D games — same math.”

Say it like this

Q

An engineer says the retrieval index uses "normalized embeddings with dot-product search." What does that imply?

MODEL ANSWER

"Normalized = all vectors scaled to length 1, so dot product equals cosine similarity — ranking is purely by direction and unaffected by document length. It also tells me any cosine-vs-dot-product debate for this index is moot."

Say it like this

Q

We want to cut vector-DB costs. What are the levers on the embedding side?

MODEL ANSWER

"Reduce dimensions (Matryoshka truncation or PCA — keep the most informative directions, drop near-redundant ones), quantize the stored vectors, or shrink the corpus via dedup. Each trades recall for cost, so gate the change on retrieval metrics against a golden set rather than eyeballing it."

Say it like this

Q

What does “dimension mismatch” mean when an engineer says a pipeline broke?

MODEL ANSWER

“Two arrays whose shapes don't line up for multiplication — e.g., a (3×4) fed into something expecting (5×p). Usually a wiring bug: wrong embedding size, wrong batch shape, or a model swap that changed output dimensions — which is why changing embedding models mid-flight requires re-indexing.”

Recap

The two moves and your new vocabulary

Things → vectors.

Then AI measures angles between them (similarity) and transforms them (matrix multiplication),
over and over.

Containers

scalar · vector · matrix · tensor · rank · shapes · dimension mismatch · transpose · y = Wx + b

Similarity

dot product · alignment · cosine similarity · Euclidean (L2) distance · normalization · attention · retrieval · recommendations

Transformation

matrix multiplication · (m×n)·(n×p)→(m×p) · composition · deep = chained · GPUs · identity · inverse · information loss

Coordinate system

vector space · basis · linear independence · orthogonality · superposition · Matryoshka / PCA

If you can explain each term with a 2-D drawing, you own this module.

NEXT

Put it in your hands

Open the Week 1 notebook:

- build vectors in NumPy,
- compute cosine similarity from scratch,
- embed 10 sentences,
- and watch the topics cluster in a heatmap

You'll have built the core of RAG retrieval in ~15 lines.

Track A, Part – 1: Linear Algebra I

TO THE NOTEBOOK ▶▶